RAPPORT TECHNIQUE

Proof of Concept — Système RAG de recommandation d'événements culturels

Projet Puls-Events — Périmètre : Brest

Auteur	Daniel
Client / commanditaire	Jérémy — Puls-Events
Type de document	Rapport technique — POC
Périmètre géographique	Brest (événements < 1 an)

1. Contexte et objectifs

Puls-Events est une plateforme web qui permet à ses utilisateurs de découvrir des événements culturels en temps réel, à partir de sources publiques comme Open Agenda. Dans l'optique d'enrichir l'expérience utilisateur, l'entreprise souhaite tester l'intégration d'un chatbot intelligent capable de fournir des recommandations d'événements personnalisées, appuyées sur des données réelles plutôt que sur les seules connaissances générales d'un modèle de langage.

Ce document présente le travail réalisé dans le cadre d'un Proof of Concept (POC) visant à démontrer la faisabilité technique d'un tel système, avant d'envisager un déploiement à plus grande échelle. La mission a été confiée avec le cadrage suivant :

- Source de données : l'API publique Open Agenda.
- Périmètre géographique : une zone au choix — Brest a été retenue pour ce POC.
- Fenêtre temporelle : uniquement des événements récents de moins d'un an (passés ou à venir).
- Le système doit pouvoir reconstruire sa base vectorielle à la demande.
- Une suite de tests unitaires doit vérifier que les données répondent aux critères métier.

Ce rapport détaille l'architecture retenue, les choix techniques effectués et leur justification, les modèles utilisés, les résultats obtenus lors des tests, ainsi que les limites actuelles du POC et les recommandations pour une version de production.

2. Architecture du système

Le système repose sur un pipeline en quatre grandes étapes, chacune correspondant à un script Python autonome et réutilisable :

Étape	Script	Rôle
1. Extraction & nettoyage	data_extract.py	Récupère les événements Open Agenda et produit un JSON propre.
2. Chunking & vectorisation	vectorize_data.py	Découpe les descriptions, génère les embeddings et construit l'index FAISS.
3. Génération augmentée (RAG)	chatbot.py	Recharge l'index, récupère le contexte pertinent et génère la réponse via Mistral.
4. Validation	test_pipeline.py	Vérifie automatiquement l'intégrité des données et des composants du pipeline.

Tableau 1 — Découpage du pipeline en scripts indépendants.

Le flux de traitement complet peut se résumer ainsi : les événements bruts d'Open Agenda sont filtrés et nettoyés, puis découpés en petits segments de texte (« chunks ») afin de préserver la précision de la recherche sémantique. Chaque chunk est transformé en vecteur numérique (embedding) via l'API Mistral, puis stocké dans un index FAISS avec ses métadonnées (titre, lieu, date). Lorsqu'un utilisateur pose une question, celle-ci est vectorisée à son tour, comparée aux

vecteurs de l'index pour retrouver les événements les plus proches sémantiquement, puis ces événements sont injectés dans le prompt envoyé au modèle de langage Mistral pour générer une réponse en langage naturel.

Cette approche — dite RAG (Retrieval-Augmented Generation) — permet au modèle de répondre à partir de données réelles et à jour, plutôt que de ses seules connaissances générales, ce qui réduit fortement le risque de réponses inventées (hallucinations).

3. Choix techniques et justifications

3.1 Source et périmètre des données

Open Agenda a été retenue car c'est la source explicitement demandée dans le cadrage du projet, et son API publique fournit des données structurées (titre, description, lieu, dates, conditions d'accès) sans nécessiter d'authentification complexe. Le périmètre a été restreint à la ville de Brest pour garder un volume de données maîtrisable pour un POC, tout en restant représentatif d'un usage réel.

3.2 Nettoyage des données

Deux règles de nettoyage ont été appliquées systématiquement :

- Suppression des événements sans titre ou sans description (dropna), ces champs étant indispensables pour générer un contenu vectorisable pertinent.
- Suppression des doublons sur l'identifiant unique de l'événement (drop_duplicates), afin d'éviter qu'un même événement pollue plusieurs fois l'index.

Le filtrage temporel (moins d'un an) est appliqué directement au niveau de la requête API, via une date calculée dynamiquement à chaque exécution plutôt qu'une date codée en dur, pour que le script reste valide dans le temps.

3.3 Découpage en chunks (chunking)

Le texte est découpé avec un RecursiveCharacterTextSplitter (LangChain), avec une taille de segment de 500 caractères et un chevauchement de 50 caractères. Un choix spécifique a été fait : seule la description de l'événement est découpée — le titre, le lieu, la date, les conditions et le public visé (l'« en-tête ») sont répétés en tête de chaque chunk issu du même événement.

Cette décision corrige un problème observé en cours de développement : lorsque l'en-tête n'était pas répété, certains chunks isolés perdaient le contexte de l'événement auquel ils appartenaient, ce qui dégradait la pertinence des résultats de recherche (par exemple, un chunk contenant uniquement le mot « gratuit » sans le titre de l'événement associé).

3.4 Vectorisation et base vectorielle

Le modèle mistral-embed est utilisé pour transformer chaque chunk en vecteur numérique. FAISS (Facebook AI Similarity Search) a été retenu comme base vectorielle pour trois raisons : il est open source et gratuit, il s'intègre nativement à LangChain, et sa simplicité d'utilisation est suffisante pour le volume de données d'un POC (quelques centaines d'événements). L'index utilisé (IndexFlatL2, exposé via le wrapper LangChain FAISS.from_documents) effectue une recherche exacte par

distance euclidienne, ce qui est le choix le plus pertinent à cette échelle — les index approximatifs (IVF, HNSW) n'apportent un gain de performance qu'à partir de volumes bien plus importants.

3.5 Orchestration avec LangChain

LangChain a été utilisé pour orchestrer l'ensemble de la chaîne de traitement (LangChain Expression Language — LCEL), depuis la recherche dans FAISS jusqu'à la génération de la réponse par le modèle de langage. Ce choix permet une chaîne déclarative, lisible et facilement modifiable (chaque étape — récupération du contexte, construction du prompt, appel au modèle, formatage de la sortie — est un maillon indépendant), tout en bénéficiant d'une intégration native avec FAISS et l'API Mistral.

3.6 Déduplication des résultats de recherche

Comme un même événement peut être représenté par plusieurs chunks, une fonction de recherche personnalisée a été développée plutôt que d'utiliser le retriever générique de LangChain : elle récupère un nombre de candidats supérieur au besoin (fetch_k), puis ne conserve qu'un chunk par identifiant d'événement unique (uid), jusqu'à atteindre le nombre de résultats souhaité (k). Cette étape s'est révélée nécessaire après un premier essai où un même événement apparaissait plusieurs fois dans les résultats renvoyés à l'utilisateur.

3.7 Génération de la réponse et sécurité

Le modèle mistral-small-latest est utilisé pour la génération, avec une température volontairement basse (0.2) afin de privilégier des réponses factuelles et fidèles au contexte plutôt que créatives. Le prompt système impose explicitement au modèle de ne répondre qu'à partir des événements fournis, de signaler l'absence de résultat plutôt que d'inventer une réponse, et de rediriger poliment l'utilisateur si la question porte sur une autre ville ou un sujet hors périmètre.

La clé API Mistral est stockée dans un fichier .env, chargé via python-dotenv, et n'apparaît jamais en dur dans le code source — une exigence de sécurité de base respectée dès la phase de développement.

4. Modèles sélectionnés

Rôle	Modèle	Justification
Vectorisation (embeddings)	mistral-embed	Modèle d'embedding officiel de Mistral, cohérent avec le reste de la pile technique et suffisant pour du texte court en français.
Génération de réponses	mistral-small-latest	Bon compromis coût / qualité pour un POC : suit bien les instructions du prompt système, latence raisonnable pour une démo live.

Tableau 2 — Modèles utilisés dans le pipeline.

5. Résultats obtenus pour le POC

5.1 Volumétrie des données

- Événements exploitables après nettoyage : 686 (sur la ville de Brest, fenêtre d'un an).

- Chunks générés après découpage : 1 245, soit une moyenne inférieure à 2 chunks par événement — cohérent avec des descriptions relativement courtes.
- Vecteurs indexés dans FAISS : 1 245 — confirmé par un contrôle automatique (vector_store.index.ntotal), garantissant qu'aucun chunk n'a été perdu lors de l'indexation.

5.2 Qualité de la recherche sémantique

Une recherche test sur la requête « concert de musique gratuit » a permis de valider deux points clés du système : la pertinence sémantique (les résultats renvoyés concernent bien des événements musicaux ou en lien avec la requête) et l'absence de doublon (trois événements distincts sont renvoyés, alors qu'un des tout premiers essais — avant la mise en place de la déduplication par uid — avait renvoyé trois fois le même événement).

#	Événement	Lieu / date
1	La Nuit des conservatoires à Brest	Conservatoire à rayonnement régional de Brest métropole — vendredi 30 janvier, 17h30
2	Cinérencontre — Soundtrack to a Coup d'État	Cinéma Les Studios — dimanche 28 septembre 2025, 16h00
3	Concert pop/rock avec Stellen	Maison ancienne années 30, Brest — mardi 14 avril, 18h00

Tableau 3 — Résultats renvoyés pour la requête « concert de musique gratuit ».

5.3 Robustesse du chatbot

Le comportement du chatbot a été vérifié sur trois types de scénarios, conformément au point de vigilance du cahier des charges :

- Question précise correspondant à un événement réel : le système retrouve et présente l'événement avec son lieu et sa date.
- Question vague ou générale (ex. « qu'est-ce qu'il y a à faire ce week-end ? ») : le système propose plusieurs événements pertinents plutôt qu'une réponse unique arbitraire.
- Question hors périmètre (autre ville, sujet non culturel) : le prompt système impose une réponse de cadrage plutôt qu'une réponse inventée, conformément à l'exigence de fiabilité du cahier des charges.

5.4 Tests unitaires automatisés

Une suite de tests (pytest) valide automatiquement l'intégrité du pipeline à chaque exécution :

- test_data_file_exists — vérifie que le fichier de données JSON a bien été généré.
- test_faiss_index_exists — vérifie que l'index FAISS a bien été créé sur disque.
- test_format_events — test unitaire pur sur la fonction de formatage du contexte injecté au LLM.
- test_events_are_recent_and_in_brest — test métier : vérifie que 100 % des événements indexés respectent les deux critères de cadrage (moins d'un an, ville de Brest).

Les quatre tests sont passants (4 passed), ce qui apporte une garantie automatisée et reproductible que les contraintes métier fixées en amont du projet sont respectées par les données effectivement utilisées.

6. Limites actuelles du POC

- Absence d'historique de conversation : chaque question est traitée indépendamment, conformément au cahier des charges du POC, mais ce n'est pas adapté à un usage conversationnel multi-tours en production.
- Absence d'évaluation quantitative formalisée : la qualité des réponses a été validée manuellement sur quelques scénarios, sans métriques systématiques (fidélité au contexte, pertinence, précision de la récupération).
- Index non incrémental : toute mise à jour des données nécessite de reconstruire entièrement l'index FAISS plutôt que de l'enrichir progressivement.
- Périmètre limité à une seule ville, sans mécanisme de filtrage dynamique par lieu ou par période au niveau de la recherche elle-même.
- Absence de seuil de similarité ou de mécanisme de re-ranking : le système retourne toujours les k résultats les plus proches, même si aucun n'est réellement pertinent.

7. Recommandations pour la version finale

- Mettre en place une évaluation continue avec un framework comme Ragas (fidélité, pertinence de la réponse, précision et rappel du contexte) pour objectiver la qualité du système et suivre son évolution dans le temps.
- Ajouter un seuil de distance minimal et/ou un mécanisme de re-ranking pour écarter les résultats faiblement pertinents avant de les injecter dans le prompt.
- Étendre le périmètre à plusieurs villes ou régions, avec un filtrage par métadonnées (lieu, période, catégorie) en complément de la recherche sémantique.
- Automatiser le rafraîchissement des données (tâche planifiée) pour que l'index reste à jour sans intervention manuelle.
- Envisager une base vectorielle plus scalable (ex. Milvus, Pinecone) ou un index FAISS approximatif si le volume de données augmente significativement.
- Ajouter une interface utilisateur (par exemple Streamlit) et une boucle de feedback utilisateur (retours positifs/négatifs) pour collecter des données d'amélioration continue.
- Réintroduire un historique de conversation pour la version de production, afin de permettre des échanges multi-tours cohérents.

8. Conclusion

Ce POC démontre la faisabilité technique d'un chatbot de recommandation d'événements culturels reposant sur une architecture RAG complète : collecte et nettoyage de données réelles, découpage et vectorisation via Mistral, indexation FAISS, et génération de réponses orchestrée par LangChain. Les tests réalisés — qu'ils soient qualitatifs (scénarios d'interaction) ou automatisés (suite pytest) — confirment que le système respecte les contraintes métier fixées (fraîcheur des données, périmètre géographique, absence de doublons) et produit des réponses pertinentes et sourcées.

Les limites identifiées sont cohérentes avec le périmètre volontairement restreint d'un Proof of Concept, et les recommandations formulées ci-dessus tracent une trajectoire claire vers une version de production plus robuste, évaluée en continu et adaptée à un usage à plus grande échelle.